Based on the CUDA concepts covered in your uploaded slides (specifically 04-cuda01.pdf, 05-cuda02.pdf, 08-mem-opt.pdf, and 09-cuda-atomics.pdf), here is a comprehensive side-by-side comparison of **CUDA C++** and **CUDA Python (Numba)**.

1. Defining a Kernel (The GPU Function)

When to use: Use this to define a function that runs on the GPU but is called from the CPU.

Feature	CUDA C++ (from slides)	Numba CUDA (Python)
Syntax	__global__ void kernel_name(args)	@cuda.jit decorator
Return	Must return void.	Must not return a value (modify arrays in place).

Example:

<table>
<tr>
<th>CUDA C++</th>
<th>Numba Python</th>
</tr>
<tr>
<td>

C++

```
//
__global__ void myKernel(int *a) {
    // Code executed on GPU
}
```

</td>
<td>

Python

```
from numba import cuda

@cuda.jit
def my_kernel(a):
    # Code executed on GPU
    pass
```

</td>
</tr>
</table>

---

2. Memory Management (Host vs. Device)

When to use: You must allocate memory on the GPU and move data from CPU (Host) to GPU (Device) before processing, and move results back afterwards.

Feature	CUDA C++	Numba CUDA (Python)
Allocate	cudaMalloc(&ptr, size)	cuda.device_array(shape) or cuda.to_device(ary)
Copy H$\to$D	cudaMemcpy(d_ptr, h_ptr, size, cudaMemcpyHostToDevice)	d_ary = cuda.to_device(h_ary)
Copy D$\to$H	cudaMemcpy(h_ptr, d_ptr, size, cudaMemcpyDeviceToHost)	h_ary = d_ary.copy_to_host()
Free	cudaFree(ptr)	Automatic (Python Garbage Collection)

Example:

<table>
<tr>
<th>CUDA C++</th>
<th>Numba Python</th>
</tr>

<tr>
<td>

C++

```
//
int *d_a;
int size = N * sizeof(int);

// 1. Allocate
cudaMalloc((void **)&d_a, size);

// 2. Copy Host to Device
cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);

// ... launch kernel ...

// 3. Copy Device to Host
cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);

// 4. Free
cudaFree(d_a);
```

</td>
<td>

Python

```
import numpy as np
from numba import cuda

h_a = np.array([...])

# 1. Allocate & 2. Copy Host to Device
# Numba handles allocation and copy in one step
d_a = cuda.to_device(h_a)

# ... launch kernel ...
```

```
# 3. Copy Device to Host
h_c = d_a.copy_to_host()

# 4. Free
# (Handled automatically when d_a goes out of scope)
```

</td>
</tr>
</table>

---

3. Thread & Block Indexing

When to use: To distinguish which part of the data each specific thread should work on. You calculate a unique global index (Global ID) for every thread.

Concept	CUDA C++	Numba CUDA (Python)
Thread ID	threadIdx.x, .y, .z	cuda.threadIdx.x, .y, .z
Block ID	blockIdx.x, .y, .z	cuda.blockIdx.x, .y, .z
Block Dim	blockDim.x, .y, .z	cuda.blockDim.x, .y, .z
Grid Dim	gridDim.x, .y, .z	cuda.gridDim.x, .y, .z
Global ID	int i = threadIdx.x + blockIdx.x * blockDim.x;	i = cuda.grid(1) (Shortcut for 1D)

Example:

<table>
<tr>
<th>CUDA C++</th>
<th>Numba Python</th>
</tr>
<tr>
<td>

```
C++
```
C++

```
//
__global__ void idxKernel(float *a) {
    // Manual calculation
    int i = threadIdx.x + blockIdx.x * blockDim.x;

    if (i < N) {
        a[i] = a[i] * 2.0f;
    }
}
```

</td>
<td>

Python

```
@cuda.jit
def idx_kernel(a):
    # Numba shortcut for:
    # threadIdx.x + blockIdx.x * blockDim.x
    i = cuda.grid(1)

    if i < a.size:
        a[i] = a[i] * 2.0
```

</td>
</tr>
</table>

---

4. Kernel Launch Syntax

When to use: To execute the kernel on the GPU. You must specify the "Grid" (number of blocks) and "Block" (threads per block) dimensions.

Feature	CUDA C++	Numba CUDA (Python)
Syntax	kernel<<<blocks, threads>>>(args)	kernel[blocks, threads](args)

Example:

<table>
<tr>
<th>CUDA C++</th>
<th>Numba Python</th>
</tr>
<tr>
<td>

C++

```
//
int threadsPerBlock = 256;
int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

// Launch
myKernel<<<blocksPerGrid, threadsPerBlock>>>(d_a);
```

</td>
<td>

Python

```
threads_per_block = 256
# Use math.ceil or integer division to calc blocks
blocks_per_grid = (N + (threads_per_block - 1)) // threads_per_block

# Launch
my_kernel[blocks_per_grid, threads_per_block](d_a)
```

</td>
</tr>
</table>

---

5. Shared Memory (Optimization)

When to use: Use shared memory when multiple threads in the same block need to access

the same data repeatedly. It is much faster than global memory but smaller and local to a block.

Feature	CUDA C++	Numba CUDA (Python)
Declaration	__shared__ int temp[SIZE];	cuda.shared.array(shape, dtype)
Scope	Visible to all threads in **one block**.	Visible to all threads in **one block**.

Example:

<table>
<tr>
<th>CUDA C++</th>
<th>Numba Python</th>
</tr>
<tr>
<td>

C++

```
//
__global__ void sharedKernel() {
    // Static allocation
    __shared__ float s_data[256];

    // Load data into shared mem
    s_data[threadIdx.x] = ...;
}
```

</td>
<td>

Python

```
from numba import float32

@cuda.jit
def shared_kernel():
    # Must specify shape and type
    s_data = cuda.shared.array(256, dtype=float32)

    # Load data into shared mem
    s_data[cuda.threadIdx.x] = ...
```

</td>
</tr>
</table>

---

6. Synchronization (Barriers)

When to use: When threads in a block communicate via shared memory, you must sync them to ensure everyone has finished writing before anyone starts reading.

Feature	CUDA C++	Numba CUDA (Python)
Syntax	__syncthreads();	cuda.syncthreads()

Example:

<table>
<tr>
<th>CUDA C++</th>
<th>Numba Python</th>
</tr>
<tr>
<td>

```
    C++
```

```
//
s_data[threadIdx.x] = input[i];

// Wait for all threads to finish writing
```

```
__syncthreads();

// Safe to read now
val = s_data[(threadIdx.x + 1) % 256];
```

</td>
<td>

Python

```
s_data[cuda.threadIdx.x] = input[i]

# Wait for all threads to finish writing
cuda.syncthreads()

# Safe to read now
val = s_data[(cuda.threadIdx.x + 1) % 256]
```

</td>
</tr>
</table>

---

7. Atomic Operations

When to use: When multiple threads try to update the *same* memory address simultaneously (e.g., calculating a histogram or sum). Without atomics, a "race condition" occurs, and data is lost.

Feature	CUDA C++	Numba CUDA (Python)
Syntax	atomicAdd(&address, val);	cuda.atomic.add(array, index, val)
Behavior	Read-Modify-Write in one uninterruptible step.	Same.

Example:

<table>

<tr>
<th>CUDA C++</th>
<th>Numba Python</th>
</tr>
<tr>
<td>

C++

```
//
__global__ void hist(int *bins, int *input) {
  int i = threadIdx.x + blockIdx.x * blockDim.x;
  int val = input[i];

  // Atomically increment the bin
  atomicAdd(&bins[val], 1);
}
```

</td>
<td>

Python

```
@cuda.jit
def hist(bins, input_arr):
  i = cuda.grid(1)
  if i < input_arr.size:
    val = input_arr[i]

    # Atomically increment bins[val]
    # Format: (array, index, value_to_add)
    cuda.atomic.add(bins, val, 1)
```

</td>
</tr>
</table>

Full "Vector Addition" Example (Side-by-Side)

This combines all the basic concepts (Kernels, Memory, Indexing, Launch).

CUDA C++:

C++

```
//
__global__ void add(int *a, int *b, int *c, int n) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    if (index < n)
        c[index] = a[index] + b[index];
}

int main() {
    int N = 1024;
    int size = N * sizeof(int);

    // Allocate Host
    int *h_a = (int*)malloc(size);
    int *h_b = (int*)malloc(size);
    int *h_c = (int*)malloc(size);
    // ... initialize h_a, h_b ...

    // Allocate Device
    int *d_a, *d_b, *d_c;
    cudaMalloc(&d_a, size);
    cudaMalloc(&d_b, size);
    cudaMalloc(&d_c, size);

    // Copy H -> D
    cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);

    // Launch
    int threads = 256;
    int blocks = (N + threads - 1) / threads;
    add<<<blocks, threads>>>(d_a, d_b, d_c, N);
```

```
    // Copy D -> H
    cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);

    // Free
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    free(h_a); free(h_b); free(h_c);
}
```

Numba CUDA:

Python

```
import numpy as np
from numba import cuda

@cuda.jit
def add(a, b, c):
    # Calculate global index (same as C++ equation)
    index = cuda.grid(1)

    # Boundary check is handled by array size property in Python
    if index < c.size:
        c[index] = a[index] + b[index]

# Main Host Code
N = 1024

# Allocate Host (NumPy arrays) & Initialize
h_a = np.arange(N, dtype=np.int32)
h_b = np.arange(N, dtype=np.int32)

# Allocate & Copy H -> D (One step in Numba)
d_a = cuda.to_device(h_a)
d_b = cuda.to_device(h_b)

# Allocate Device Output (Empty)
d_c = cuda.device_array(N, dtype=np.int32)

# Launch Configuration
threads = 256
```

```
blocks = (N + (threads - 1)) // threads
add[blocks, threads](d_a, d_b, d_c)

# Copy D -> H
h_c = d_c.copy_to_host()

# Freeing is automatic in Python
```

---

1. Global Memory (The Standard)

What it is: The main, large DRAM on the GPU. It is slow (high latency) but huge.
When to use: For all your main input/output data. Both C++ and Numba default to this when you allocate "Device" memory.

Feature	**CUDA C++**	**Numba CUDA (Python)**
Declaration	Standard pointer float *d_ptr allocated via cudaMalloc	cuda.device_array() or cuda.to_device()
Access	val = d_ptr[i];	val = d_ary[i]
Performance Note	Must be **Coalesced** (threads read consecutive addresses)	Same. Numba arrays are row-major (C-style) by default, matching hardware preference.

---

2. Pinned Memory (Page-Locked Host Memory)

What it is: Host (CPU) memory that the OS cannot swap to disk.
Why use it: It allows faster (up to 2x) transfer speeds (PCIe DMA) between CPU and GPU compared to standard "pageable" memory.

Feature	**CUDA C++**	**Numba CUDA (Python)**
Allocation	cudaMallocHost(&ptr, size);	ary = cuda.pinned_array(shape, dtype=...)
Usage	Use it as the *source* or *dest*	Use it like a normal NumPy

	in cudaMemcpy.	array on the CPU.
Freeing	cudaFreeHost(ptr);	Automatic (when object goes out of scope).

Example:

<table>
<tr>
<th>CUDA C++</th>
<th>Numba Python</th>
</tr>
<tr>
<td>

C++

```
int *h_pinned;
// Allocate pinned memory on CPU
cudaMallocHost((void**)&h_pinned, bytes);

// Copy is faster now
cudaMemcpy(d_data, h_pinned, bytes, ...);
```

</td>
<td>

Python

```
# Create a pinned numpy-like array
h_pinned = cuda.pinned_array(1024, dtype=np.int32)

# Fill it
h_pinned[:] = np.arange(1024)

# Transfer is faster
d_data = cuda.to_device(h_pinned)
```

</td>
</tr>
</table>

---

3. Unified Memory (Managed Memory)

What it is: A single pointer accessible from both CPU and GPU. The driver automatically migrates pages of data to where they are needed.
When to use: To simplify code (no explicit cudaMemcpy needed) or to handle datasets larger than GPU memory.

Feature	CUDA C++	Numba CUDA (Python)
Allocation	cudaMallocManaged(&ptr, size);	ary = cuda.managed_array(shape, dtype=...)
Migration	Automatic (Driver handles page faults).	Automatic.
Synchronization	cudaDeviceSynchronize() needed after kernel before CPU reads.	cuda.synchronize() needed after kernel before CPU reads.

Example:

<table>
<tr>
<th>CUDA C++</th>
<th>Numba Python</th>
</tr>
<tr>
<td>

C++

```
int *data;
// Allocate Unified Memory
```

```
cudaMallocManaged(&data, size);

// Initialize on CPU
data[0] = 10;

// Run on GPU (no copy needed!)
kernel<<<1, 1>>>(data);

// Wait for GPU to finish
cudaDeviceSynchronize();

// Read result on CPU
printf("%d", data[0]);
```

</td>
<td>

Python

```
# Allocate Unified Memory
data = cuda.managed_array(1024, dtype=np.int32)

# Initialize on CPU
data[0] = 10

# Run on GPU (no explicit copy needed!)
kernel[1, 1](data)

# Wait for GPU to finish
cuda.synchronize()

# Read result on CPU
print(data[0])
```

</td>
</tr>
</table>

---

4. Constant Memory

What it is: A small (64KB), read-only memory cache optimized for when all threads read the

same address simultaneously (broadcasting).
When to use: For coefficients, lookup tables, or simulation parameters that don't change during the kernel.

Feature	CUDA C++	Numba CUDA (Python)
Declaration	__constant__ float c_data[SIZE]; (Global scope)	ary = cuda.const.array_like(numpy_ary) (Inside kernel)
Copy H$\to$D	cudaMemcpyToSymbol(c_data, h_src, size);	Passed implicitly or handled by Numba.
Scope	File scope.	Inside the kernel function.

Example:

<table>
<tr>
<th>CUDA C++</th>
<th>Numba Python</th>
</tr>
<tr>
<td>

C++

```
// Declared at global scope
__constant__ float FACTS[5];

int main() {
    float h_facts[] = {1.1, 2.2, ...};
    // Special copy function
    cudaMemcpyToSymbol(FACTS, h_facts, sizeof(h_facts));
    kernel<<<...>>>();
}

__global__ void kernel() {
    // Fast if all threads read same index
    float f = FACTS[0];
```

```
}
```

</td>
<td>

Python

```
# Define constant data
FACTS = np.array([1.1, 2.2, ...])

@cuda.jit
def kernel(output):
    # Declare 'c_facts' as constant memory
    # matching 'FACTS' content/shape
    c_facts = cuda.const.array_like(FACTS)

    idx = cuda.grid(1)
    output[idx] = c_facts[0] # Fast access
```

</td>
</tr>
</table>

---

5. CUDA Streams (Asynchronous Execution)

What it is: Queues of operations. Operations in the same stream run sequentially; operations in different streams can run concurrently (overlap).
When to use: To overlap data transfers (Copy) with computation (Kernel execution) to hide latency.

Feature	CUDA C++	Numba CUDA (Python)
Create	cudaStream_t s; cudaStreamCreate(&s);	s = cuda.stream()
Use in Copy	cudaMemcpyAsync(..., stream)	cuda.to_device(..., stream=s)
Use in Kernel	kernel<<<..., stream>>>(...)	kernel[..., stream=s](...)

Sync	cudaStreamSynchronize(s);	s.synchronize()

Example (Overlapping Copy & Execute):

<table>
<tr>
<th>CUDA C++</th>
<th>Numba Python</th>
</tr>
<tr>
<td>

C++

```
cudaStream_t s1;
cudaStreamCreate(&s1);

// Async Copy (must use Pinned Memory for overlap!)
cudaMemcpyAsync(d_a, h_a, size, H2D, s1);

// Launch Kernel in same stream
kernel<<<grid, block, 0, s1>>>(d_a);

// CPU continues immediately...
cudaStreamSynchronize(s1);
```

</td>
<td>

Python

```
s1 = cuda.stream()

# Async Copy (must use Pinned Memory for overlap!)
# Note: copy_to_device is the explicit method
d_a = cuda.to_device(h_a, stream=s1)

# Launch Kernel in same stream
```

```
kernel[grid, block, s1](d_a)

# CPU continues immediately...
s1.synchronize()
```

</td>
</tr>
</table>